Übung 9 - TypeScript 2

Aufgabe 1 (**Funktionen in TypeScript**)

(a) Berechnen Sie die Fibonacci-Funktion mit einer rekursiven Funktion. Zusatzfrage: Wie oft wird die Funktion fib aufgerufen, um z.B. fib(5) zu berechnen?

(b) Berechnen Sie den größten gemeinsamen Teiler (ggT) mit einer rekursiven Funktion.

(c) Schreiben Sie eine rekursive Funktion, die die Collatz-Folge überprüft. Diese Folge ist wie folgt definiert:
c(n)=n/2, falls n gerade
c(n)=3*n+1, falls n ungerade.
Behauptung: Die Folge endet immer mit 1.
Geben Sie jedes Folgenglied aus. Testen Sie mit 5, 17 und 23.

(d) Auch in der Mathematik gibt es Lambda-Funktionen, wie z.B. $\lambda x.x+1$. D.h. die Funktion ohne Namen, die zu einem Argument eins hinzuaddiert. Definieren Sie die folgende Lambda-Funktion in Typescript: $\lambda x.\lambda y.x * y - x - y$.

(e) Wie wird der Mehrwertsteuersatz in der Funktion berechneMehrwertsteuer behandelt, wenn kein zweiter Parameter übergeben wird?

```
1 function berechneMehrwertsteuer(preis: number, satz: number =
      0.19): number {
2     return preis * satz;
3 }
4
5 let nettopreis = 50;
6 let bruttopreis = nettopreis +
      berechneMehrwertsteuer(nettopreis);
```

(f) Schreiben Sie eine Funktion, welche eine unbestimmte Anzahl an Parametern (Zahlen) annehmen kann und die Summe der übergebenen Argumente zurückgibt. (Beispiel: sum(13, 21, 8) → 42 aber auch sum(1, 8, 2, 19, 30) → 60).

Aufgabe 2 (**Klassen**)

Erstellen Sie eine Klasse `Book` in TypeScript, die folgende Eigenschaften und Methoden besitzt:

- Zwei Eigenschaften: `title: string` und `author: string`
- Einen Konstruktor, der beide Eigenschaften initialisiert
- Eine Methode `describe()`, die einen Text der Form `"Das Buch 'Der Titel' wurde von Der Autor geschrieben."` zurückgibt

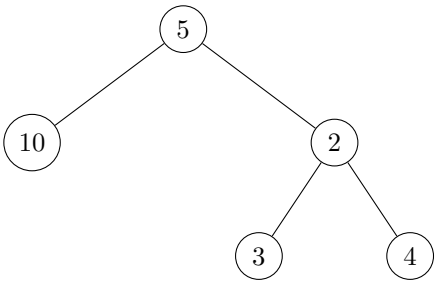
Erzeugen Sie anschließend zwei Objekte dieser Klasse und geben Sie deren Beschreibung mit `console.log()` aus.

Aufgabe 3 (**Abstrakte Klassen**)

Modellieren Sie einen binären gerichteten azyklischen Graphen, d.h. einen binären Baum, indem Sie die folgende abstrakte Klasse `TreeNode` um zwei konkrete Klassen `LeafNode` (Blätter) und `InternalNode` (innere Knoten) erweitern. Die Funktion `calculateValue` gibt bei einem Blatt einen `value` zurück und bei inneren Knoten die Summe aus Value des linken und des rechten Teilbaums multipliziert mit dem Gewicht (`weight`) Wert des Knotens.

```
1 abstract class TreeNode {
2     abstract calculateValue(): number;
3 }
```

Beispiel:



Der Wert dieses Baumes ist: $(10 + (3 + 4) * 2) * 5 = (10 + 14) * 5 = 120$.